



Rangkuman Kelas



a dan Bilqis akhirnya menyelesaikan mata kuliah Machine Learning dengan nilai sempurna, menjadikan mereka murid terbaik di kelas tersebut. Perjalanan mereka tidaklah mudah; dari awal, kedua sahabat ini menghadapi berbagai tantangan, mulai dari memahami konsep dasar hingga menerapkan algoritma kompleks dalam proyek-proyek praktikum.

Diana, dengan ketekunan dan rasa ingin tahunya yang tinggi, selalu menjadi yang pertama datang ke kelas dan terakhir meninggalkan laboratorium. Ia rajin mencari referensi tambahan dan sering berdiskusi dengan dosen untuk memperdalam pemahamannya. Sementara itu, Bilqis mengandalkan kemampuan analitisnya yang tajam dan kreativitasnya dalam memecahkan masalah. Bersama-sama, mereka membentuk tim yang solid, saling melengkapi kekuatan masing-masing.

Kerja keras mereka membuahkan hasil ketika proyek akhir mereka diakui sebagai yang terbaik di kelas, tidak hanya mendapatkan nilai sempurna, tetapi juga diapresiasi oleh dosen dan rekan-rekan mereka. Keberhasilan Diana dan Bilqis menjadi inspirasi bagi banyak mahasiswa lainnya yang ingin menguasai bidang Machine Learning.

Tentunya, Anda juga tidak mau kalah, 'kan? Selagi masih ada waktu, Anda perlu mengejar kesuksesan seperti mereka berdua. Dengan menyelesaikan kelas ini, Anda semakin dekat dengan pencapaian yang telah direncanakan. Setiap tantangan yang Anda hadapi adalah langkah menuju keahlian yang lebih tinggi dan peluang karier yang lebih gemilang.

Oleh karena itu, mari kita selesaikan kelas ini dengan sangat sempurna. Jadikan setiap sesi belajar sebagai kesempatan untuk tumbuh dan berkembang. Ingatlah bahwa kesuksesan Diana dan Bilqis dimulai dari tekad serta kerja keras mereka. Semangat terus dalam belajar dan jangan ragu untuk terus berusaha. Kesuksesan ada di depan mata, dan Anda juga bisa mencapainya, mari kita selesaikan rangkuman kelas Belajar Machine Learning untuk Pemula!





ngantar Machine Learning



Salah satu dari beberapa dekade terakhir, machine learning telah mengalami kemajuan pesat dan menjadi pusat dari banyak inovasi teknologi. Ini memegang peranan penting dalam revolusi industri 4.0 yang dipenuhi dengan perubahan digital besar-besaran.

Machine learning tidak hanya mendukung produk-produk canggih, tetapi juga mengubah cara kita berinteraksi dengan teknologi, meningkatkan efisiensi operasional, dan membuka peluang baru dalam berbagai sektor industri.

Apa Itu Machine Learning?

Menurut Arthur Samuel, seorang pionir dalam kecerdasan buatan, machine learning adalah berikut.

"A field of study that gives computers the ability to learn without being explicitly programmed."

(Arthur Samuel, 1959)

Arthur Samuel memperkenalkan istilah machine learning pada tahun 1959. Dia mendefinisikan machine learning sebagai cabang ilmu yang memungkinkan komputer belajar dan berkembang tanpa diprogram secara eksplisit untuk setiap tugas. Jadi, alih-alih memerlukan serangkaian aturan dari manusia, mesin dapat mempelajari pola berdasarkan data. Mesin pun dapat membuat keputusan atau prediksi berdasarkan pembelajaran tersebut. Ini membuat proses pemrograman lebih fleksibel dan adaptif.

Taksonomi Kecerdasan Buatan

Untuk memahami machine learning lebih baik, kita perlu melihat taksonomi kecerdasan buatan (AI). Berikut adalah urutan taksonomi AI secara sederhana.

1. Artificial Intelligence (AI)

AI adalah konsep dasar mencakup penggunaan komputer atau mesin untuk melakukan tugas yang biasanya membutuhkan kecerdasan manusia, seperti pengambilan keputusan dan pemecahan masalah.

2. Machine Learning (ML)

Machine learning adalah cabang dari AI yang memungkinkan komputer belajar dari data tanpa diprogram secara eksplisit. ML membantu komputer mengenali pola dan membuat prediksi berdasarkan data yang telah dipelajari.

3. Neural Network (NN)

Neural network adalah model matematis terinspirasi dari jaringan saraf manusia. Pada ML, NN digunakan untuk memproses informasi dan belajar dari data dengan menghubungkan neuron-neuron buatan dalam lapisan-lapisan.

4. Deep Learning (DL)

Deep learning adalah sub-bidang ML yang menggunakan neural network dengan banyak lapisan (deep neural





Generative AI adalah cabang AI yang fokus pada pembuatan konten baru dan orisinal, seperti teks, gambar, serta musik. Berbeda dari AI tradisional yang mengikuti aturan yang sudah ada, generative AI dapat menghasilkan sesuatu yang belum pernah ada sebelumnya.

Komponen Utama dalam Machine Learning

Berikut adalah komponen utama dalam machine learning yang penting untuk membangun model yang efektif.

1. Data

Data adalah bahan dasar untuk machine learning. Tanpa data, model tidak dapat dilatih. Data terdiri dari fitur (atribut) dan label (hasil yang ingin diprediksi).

2. Model

Model adalah algoritma matematis yang digunakan untuk mempelajari pola dari data. Misalnya, model regresi linier atau decision tree.

3. Algoritma

Algoritma adalah metode yang digunakan untuk melatih model dengan data. Algoritma menentukan cara model belajar dari data dan mengoptimalkan prosesnya.

4. Feature Engineering

Feature engineering adalah proses mengubah data mentah menjadi fitur yang lebih relevan dan informatif untuk model. Ini termasuk pemilihan dan pembuatan fitur baru.

5. Training

Training adalah proses model belajar dari data. Model memperbarui parameter untuk meminimalkan kesalahan dan meningkatkan akurasi.

6. Evaluation

Evaluation adalah proses menilai kinerja model menggunakan data yang tidak digunakan selama pelatihan. Ini melibatkan metrik evaluasi, seperti akurasi dan precision.

7. Hyperparameter Tuning

Hyperparameter tuning adalah proses mengoptimalkan parameter di luar model yang memengaruhi kinerja model. Ini dilakukan untuk meningkatkan akurasi model.

8. Deployment

Deployment adalah tahap akhir ketika model yang telah dilatih diterapkan dalam lingkungan nyata untuk digunakan pada aplikasi atau sistem.

Jenis-Jenis Machine Learning

Ada beberapa jenis machine learning, masing-masing dengan pendekatan dan tujuan yang berbeda.





2. Unsupervised Learning

Unsupervised learning menggunakan data tanpa label untuk mengidentifikasi pola atau struktur dalam data.

Ini termasuk teknik seperti clustering, yakni mengelompokkan data berdasarkan fitur yang ada.



Semi-Supervised Learning

Semi-supervised learning menggabungkan data berlabel dan tidak berlabel. Model dilatih dengan data berlabel dan kemudian menggunakan data tidak berlabel untuk meningkatkan akurasi.

4. Reinforcement Learning

Reinforcement learning fokus pada cara agen belajar mengambil tindakan untuk memaksimalkan hadiah.

Agen belajar melalui percobaan dan kesalahan untuk menemukan tindakan terbaik dalam situasi tertentu.

Merumuskan Masalah dalam Machine Learning

Merumuskan masalah dalam machine learning adalah langkah awal yang penting dalam pengembangan model.

Berikut langkah-langkahnya.

1. Identifikasi Tujuan Bisnis

- **Tujuan:** Menetapkan hal yang ingin dicapai (misalnya, meningkatkan penjualan, mengurangi *churn*).
- **Metode SMART:** Menetapkan tujuan yang spesifik, terukur, dapat dicapai, relevan, dan terikat waktu.

2. Pahami Data yang Tersedia

- **Inventarisasi Data:** Identifikasi semua sumber data internal dan eksternal.
- **Evaluasi Kualitas Data:** Periksa kelengkapan, konsistensi, keakuratan, dan relevansi data.
- **Eksplorasi Data:** Gunakan visualisasi serta analisis statistik untuk memahami pola dan hubungan antar variabel.
- **Pemilihan Fitur:** Pilih fitur yang relevan untuk model.
- **Pembersihan dan Transformasi Data:** Atasi masalah, seperti missing values, duplikasi, dan transformasi data untuk mempersiapkannya.

3. Tentukan Jenis Masalah Machine Learning

- **Klasifikasi:** Mengelompokkan data ke dalam kategori (misalnya, menentukan apakah pelanggan akan *churn*).
- **Regresi:** Memprediksi nilai numerik kontinu (misalnya, tinggi badan).
- **Clustering:** Mengelompokkan data berdasarkan kesamaan tanpa label (misalnya, segmentasi pelanggan).

4. Definisikan Variabel Target dan Fitur

- **Fitur:** Variabel yang digunakan untuk mendeskripsikan data (misalnya, panjang dan lebar sepal bunga iris).
- **Target:** Variabel yang ingin diprediksi atau diklasifikasikan (misalnya, spesies bunga iris).





ntangan dalam Machine Learning



Kualitas Data: Data harus bersih dan bebas dari kesalahan; data yang buruk dapat memengaruhi akurasi model.



2. **Keterbatasan Data:** Terlalu sedikit data atau ketidakseimbangan kategori dapat memengaruhi efektivitas model.
3. **Pemilihan Model dan Algoritma:** Memilih model dan algoritma yang tepat serta mengatur hyperparameter dengan benar.
4. **Etika dan Privasi:** Melindungi data pribadi dan menghindari bias dalam data.
5. **Pemeliharaan dan Pembaruan:** Model perlu diperbarui secara berkala agar tetap relevan dengan data terbaru.





ke modul berikutnya.



Pendahuluan Machine Learning Workflow

ⓧ kerja machine learning menurut buku tersebut melibatkan serangkaian langkah yang sistematis dan iteratif. Setiap langkah membutuhkan evaluasi dan penyesuaian untuk memastikan model yang dihasilkan dapat memberikan hasil yang akurat dan andal dalam lingkungan produksi. Buku ini juga menekankan pentingnya pemahaman mendalam terhadap data dan masalah yang dihadapi serta penggunaan alat dan teknik yang tepat untuk mencapai tujuan machine learning.

Proses Pengumpulan Data

Langkah pertama dalam alur kerja machine learning adalah memahami masalah yang ingin diselesaikan dan tujuan bisnis yang ingin dicapai. Setelah masalah dipahami, langkah berikutnya adalah mengumpulkan data yang relevan dari berbagai sumber, seperti basis data, API, atau data publik.

Exploratory Data Analysis

Setelah data terkumpul, kita perlu memahami struktur dan karakteristik data. Ini termasuk melakukan analisis deskriptif dan visualisasi data untuk menemukan pola atau anomali tertentu.

Tahapan tersebut disebut exploratory data analysis atau EDA yang bertujuan sebagai analisis awal terhadap data dan melihat bagaimana kualitas data untuk meminimalkan potensi kesalahan di kemudian hari. Pada proses ini dilakukan investigasi awal pada data untuk memahami data, menemukan pola, anomali, menguji hipotesis, memahami distribusi, frekuensi, hubungan antara variabel, dan memeriksa asumsi dengan teknik statistik dan representasi grafik.

Pada umumnya, EDA dilakukan dengan dua cara, yaitu univariate analysis, dan multivariate analysis. Univariate analysis adalah analisis deskriptif yang memeriksa pola dengan satu variabel pada modelnya. Multivariate analysis merupakan analisis deskriptif yang memeriksa pola dalam data multidimensi dengan mempertimbangkan dua atau lebih variabel. Jika terdapat dua variabel yang akan dianalisis, ia disebut bivariate analysis.

Data Preprocessing

Data preprocessing adalah langkah penting dalam alur kerja machine learning yang bertujuan untuk mempersiapkan data mentah agar dapat digunakan secara efektif oleh model machine learning.

Proses ini mencakup serangkaian teknik dan transformasi untuk memastikan data yang digunakan berkualitas tinggi, konsisten, dan relevan dengan tujuan analisis atau pemodelan. Dengan kata lain, proses ini mengubah dan mentransformasi fitur-fitur data ke dalam bentuk yang mudah diinterpretasikan dan diproses oleh algoritma machine learning.

Model Selection

Model selection adalah langkah penting dalam alur kerja machine learning yang melibatkan pemilihan algoritma





Setelah menentukan kategori utama yang cocok untuk permasalahan yang ingin diselesaikan, Anda perlu melakukan eksplorasi algoritma untuk kategori yang sudah ditentukan.

Di tahap ini, Anda perlu melakukan pengujian awal atau *baseline model* dengan tujuan untuk menguji beberapa algoritma secara cepat sehingga mendapatkan gambaran awal tentang kinerja mereka pada dataset. Hal ini bisa dilakukan dengan menggunakan *default hyperparameters* tanpa penyesuaian khusus. Hasil baseline ini membantu Anda dalam mengevaluasi seberapa kompleks atau sederhana model yang diperlukan.

- **Model Evaluation**

Setelah mengotak-atik model Anda dengan hyperparameter yang berbeda, akhirnya Anda mendapatkan model yang kinerjanya cukup baik. Langkah selanjutnya adalah mengevaluasi model akhir pada data uji.

- **Deployment**

Model deployment adalah salah satu langkah terpenting dalam alur kerja machine learning. Model yang telah dilatih dan diuji diterapkan ke dalam lingkungan produksi sehingga dapat digunakan oleh pengguna akhir atau sistem untuk membuat prediksi pada data baru.

Proses ini tidak hanya mencakup memindahkan model dari fase pengembangan ke fase produksi, tetapi juga memastikan bahwa model berjalan dengan andal, cepat, dan aman dalam skala besar.

- **Monitoring**

Ketika sebuah model machine learning sudah diterapkan dalam produksi dan digunakan oleh pengguna, pekerjaan belum selesai. Model ini masih perlu terus dipantau untuk memastikan bahwa ia tetap berfungsi dengan baik dan memberikan hasil yang akurat.

Hal ini sangat penting karena data yang dihadapi oleh model dalam tahap produksi bisa berbeda dari data yang digunakan saat model dilatih. Jika model bertemu dengan data yang tidak dikenali atau berbeda dari pola yang telah dipelajari, performanya bisa menurun seiring berjalannya waktu.

Pengenalan Tools dan Library Populer pada Python untuk Machine Learning dan Data Science

Library pada Python adalah kumpulan modul yang berisi kode-kode fungsional yang telah ditulis sebelumnya dan dapat digunakan kembali untuk menyelesaikan berbagai tugas. Modul ini bisa mencakup fungsi, kelas, dan variabel yang dapat diimpor ke dalam program Python Anda untuk mempermudah pengembangan perangkat lunak.

Library akan sangat membantu Anda untuk menyelesaikan tugas-tugas dengan lebih efisien dan efektif dengan menyediakan fungsi-fungsi siap pakai yang bisa langsung digunakan tanpa perlu menulis ulang kode dari awal.

Tools untuk Pemrograman Python





menulis kode, mengeksekusinya, dan melihat hasilnya dalam satu platform yang terpadu. Anda juga bisa membuat konfigurasi, mengatur *workflow*, dan mengelola proyek data science Anda dengan cara yang lebih intuitif.



Jupyter Notebook



Jupyter Notebook adalah perangkat lunak gratis, *open-source*, dan layanan web yang mendukung berbagai bahasa pemrograman, termasuk Python.



Google Colaboratory

Selanjutnya, mari kita kenalan dengan Google Colaboratory, atau yang sering disebut Colab—alat revolusioner yang memungkinkan Anda menulis dan menjalankan kode Python langsung melalui browser.

- **IBM Watson Studio**

IBM Watson Studio merupakan salah satu layanan dari IBM yang banyak digunakan oleh analis dan ilmuwan data. Anda juga dapat menjalankan kode secara online pada layanan seperti IBM Watson Studio tanpa perlu instal perangkat lunak apa pun pada komputer. Sebelum menggunakan IBM Watson Studio, buatlah akun IBM Cloud terlebih dahulu. Akun IBM Cloud dapat dipakai untuk mengakses IBM Watson Studio, IBM Watson Machine Learning, dan IBM Cloud.

Library Populer untuk Machine Learning dan Data Science

Python telah menjadi bahasa pilihan bagi banyak data scientist dan engineer karena kekayaan ekosistem library yang dimilikinya. Library ini mempermudah berbagai aspek dalam data science dan machine learning, mulai dari manipulasi data, visualisasi, hingga pengembangan model machine learning. Dalam materi ini, kita akan membahas beberapa library Python yang paling populer dan sering digunakan dalam bidang ini.

- **NumPy**

[NumPy](#) adalah library dasar untuk komputasi numerik di Python. Library ini menyediakan dukungan untuk array multidimensi dan berbagai fungsi matematika yang cepat dan efisien.

- **Pandas**

[Pandas](#) adalah library yang digunakan untuk manipulasi dan analisis data, terutama data yang terstruktur dalam bentuk tabel. Dengan Pandas, Anda dapat dengan mudah melakukan operasi filtering, agregasi, dan manipulasi data lainnya.

- **Matplotlib**

[Matplotlib](#) adalah sebuah library untuk membuat plot atau visualisasi data dalam 2 dimensi. Matplotlib mampu menghasilkan grafik dengan kualitas tinggi. Matplotlib dapat dipakai untuk membuat plot seperti histogram, scatter plot, grafik batang, pie chart, hanya dengan beberapa baris kode. Library ini sangat ramah pengguna.

- **Scikit Learn**

[Scikit-Learn](#) adalah library machine learning yang menyediakan berbagai algoritma pembelajaran mesin serta alat untuk preprocessing data, evaluasi model, dan tuning hyperparameter.

- **TensorFlow**





learning.

- **PyTorch**



Dikembangkan oleh Facebook, [PyTorch](#) adalah library yang dapat dipakai untuk masalah ML, computer vision, hingga pemrosesan bahasa alami. Bersaing dengan TensorFlow khususnya sebagai framework machine learning, PyTorch lebih populer di kalangan akademisi dibanding TensorFlow. Namun, dalam industri, TensorFlow lebih populer karena skalabilitasnya lebih baik dibanding PyTorch.

- **Keras**

[Keras](#) adalah library deep learning yang luar biasa. Salah satu faktor yang membuat Keras sangat populer adalah penggunaannya yang minimalis dan simpel dalam mengembangkan deep learning. Keras dibangun di atas TensorFlow yang menjadikan Keras sebagai API dengan level lebih tinggi (Higher level API) dari TensorFlow sehingga antarmukanya lebih mudah dari TensorFlow. Keras sangat cocok untuk mengembangkan model deep learning dengan waktu yang lebih singkat atau untuk pembuatan prototipe.

- **NLTK dan SpaCy**

[NLTK](#) (Natural Language Toolkit) dan SpaCy adalah library untuk Natural Language Processing (NLP). NLTK lebih banyak digunakan dalam riset dan pendidikan, sedangkan SpaCy lebih fokus pada kecepatan dan produksi.

- **SciPy**

[SciPy](#) adalah library yang menyediakan berbagai algoritma dan fungsi untuk komputasi ilmiah dan teknik. SciPy sering digunakan bersama NumPy untuk operasi yang lebih kompleks. Fungsi utama dari library ini untuk melakukan optimisasi, integrasi, dan pemecahan persamaan diferensial serta pengolahan sinyal dan gambar, serta analisis statistik. Berikut adalah contoh penggunaan library SciPy.

Data Collecting

Dalam machine learning, data collecting adalah fondasi tersebut. Tanpa data yang tepat dan berkualitas, model machine learning Anda tidak akan mampu berdiri kokoh, apalagi memberikan hasil yang andal.

Data collecting adalah langkah pertama dalam alur kerja machine learning di mana Anda mengumpulkan semua informasi yang dibutuhkan untuk melatih model. Data ini bisa datang dari berbagai sumber dan dalam berbagai bentuk—mulai dari data numerik, teks, gambar, hingga data kategori. Kualitas dan kuantitas data yang Anda kumpulkan akan sangat menentukan performa model machine learning yang Anda bangun.

Data Loading

Loading dataset dalam konteks machine learning adalah proses mengimpor atau memasukkan data ke dalam lingkungan pemrograman atau sistem yang digunakan untuk pengembangan model machine learning. Dataset ini berfungsi sebagai input yang akan digunakan oleh model untuk belajar dan membuat prediksi.

Proses loading dataset biasanya mencakup pengambilan data dari sumber eksternal (seperti file CSV, database,





Data Cleaning dan Transformation

Data Cleaning atau pembersihan data adalah proses penting dalam alur kerja machine learning yang bertujuan untuk meningkatkan kualitas dataset sebelum digunakan untuk pelatihan model. Dalam konteks machine learning, data cleaning mencakup serangkaian langkah yang dirancang untuk mendeteksi, memperbaiki, atau menghapus data yang tidak valid, tidak lengkap, tidak akurat, atau tidak relevan. Silakan simak gambar berikut dan perhatikan detailnya saksama.

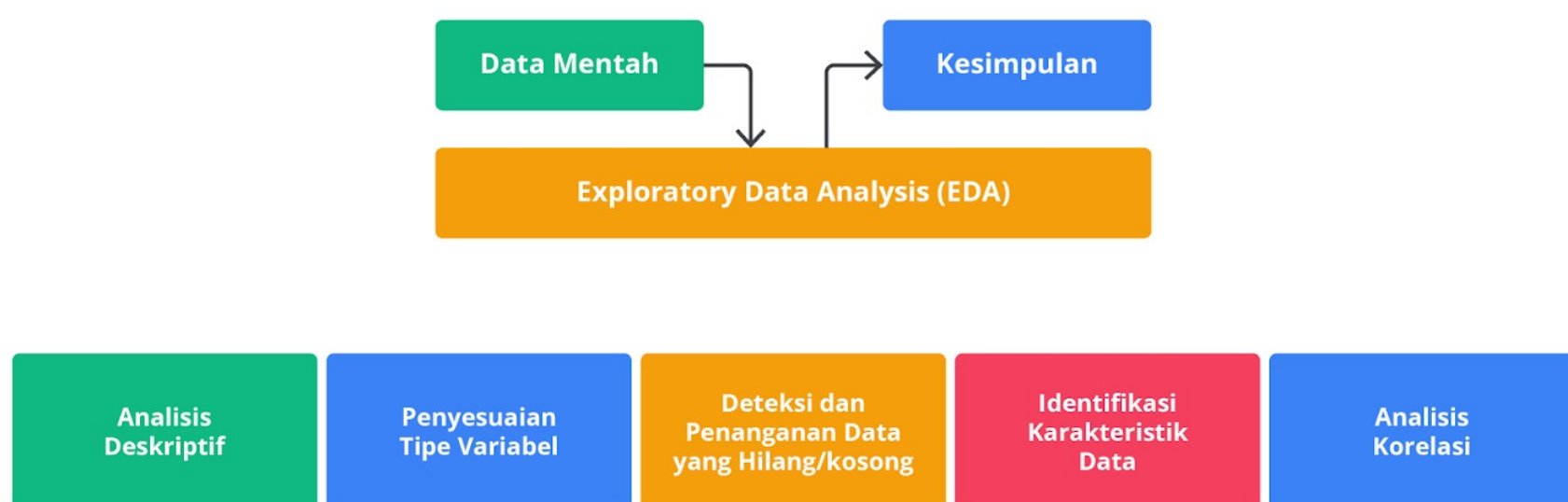
#	Id	Nama	Tanggal Lahir	Jenis Kelamin	Guru (True/False)	Siswa	Negara	Kota
1	111	John	31/12/1990	M	0	0	Ireland	Dublin
2	222	Mery	15/10/1979	F	1	15	Iceland	
3	333	Alice	19/04/2000	F	0	0	Spain	Madrid
4	444	Mark	01/11/1997	M	0	0	France	Paris
5	555	Alex	15/03/2000	A	1	23	Germany	Berlin
6	555	Peter	1983-12-01	M	1	10	Italy	Rome
7	777	Calvin	05/05/1995	M	0	0	Italy	Italy
8	888	Roxane	03/09/1948	F	0	0	Portugal	Lisbon
9	999	Anne	05/09/1992	F	0	5	Switzerland	Geneva
10	101010	Paul	14/11/1992	M	1	26	Ytali	Rome

Annotations:

- Missing Values: Empty cell for Kota in row 2.
- Invalid Values: 'A' for Jenis Kelamin in row 5.
- Misfielded Values: 'Italy' for Kota in row 7.
- Uniqueness: Duplicate Id values (555) in rows 5 and 6.
- Formats: Inconsistent date formats (DD/MM/YYYY vs YYYY-MM-DD) in rows 3 and 6.
- Attribute Dependencies: Inconsistent values for Guru/Siswa (0/1) in row 9.
- Typo: 'Ytali' for Negara in row 10.

Exploratory dan Explanatory Data Analysis

EDA merupakan tahap eksplorasi data yang telah dibersihkan guna memperoleh insight dan menjawab pertanyaan analisis. Pada prosesnya, kita akan sering menggunakan berbagai teknik dan parameter dalam *descriptive statistics* yang bertujuan untuk menemukan pola, hubungan, serta membangun intuisi terkait data yang diolah. Selain itu, tidak jarang kita juga menggunakan visualisasi data untuk menemukan pola dan memvalidasi parameter descriptive statistics yang diperoleh.





dengan data guna menemukan pola, mendapatkan wawasan, menjawab berbagai tantangan bisnis serta menyusun kesimpulan berdasarkan hasil analisis yang diperoleh.



Exploratory Data Analysis (EDA) dan Explanatory Data Analysis (ExDA) adalah dua tahap yang penting dalam proses analisis data, namun memiliki tujuan dan pendekatan yang berbeda. Mari kita *breakdown* kedua tahap tersebut agar komprehensif mulai dari tujuan, metodologi, visualisasi data, audiens, contoh kasus hingga outputnya.

Exploratory Data Analysis (EDA):

- Tujuan utama dari EDA adalah untuk memahami struktur, karakteristik, dan pola dalam data. Pada tahap ini, Anda memiliki misi untuk menemukan insight atau informasi yang tersembunyi dalam data, mengidentifikasi anomali, serta memahami hubungan antar variabel.
- EDA bersifat eksploratif dan terbuka sehingga Anda tidak memiliki hipotesis yang pasti di awal prosesnya. Sebaliknya, Anda dapat menggunakan EDA untuk membangun hipotesis atau memahami lebih dalam data yang mereka miliki.

Explanatory Data Analysis (ExDA):

- ExDA di sisi lain memiliki tujuan utama untuk mengomunikasikan temuan atau insight yang sudah didapatkan kepada audiens yang lebih luas, seperti stakeholder, tim eksekutif, atau klien.
- Pada tahap ini, analisis data berfokus pada penyampaian informasi yang jelas, ringkas, dan meyakinkan, dengan dukungan visualisasi yang efektif dan narasi yang kuat.

Jika disimpulkan, explanatory analysis merupakan proses penyampaian temuan menarik dari proses exploratory analysis. Proses penyampaian ini tentunya harus diikuti dengan visualisasi data yang baik dan efektif.

Data Splitting

Data Splitting adalah proses membagi dataset menjadi beberapa subset yang terpisah untuk tujuan pelatihan, validasi, dan pengujian model machine learning. Proses ini merupakan langkah penting dalam pipeline machine learning karena membantu memastikan model yang dikembangkan mampu membuat prediksi yang baik tidak hanya pada data pelatihan, tetapi juga pada data baru yang belum pernah dilihat sebelumnya.

Deployment dan Monitoring

Setelah model machine learning dilatih, diuji, dan dioptimalkan, langkah berikutnya adalah *deployment* (penerapan) dan *monitoring* (pemantauan) dari model tersebut. Hal ini menjadi penting dikarenakan tahapan ini satu-satunya cara agar model yang Anda bangun sebelumnya dapat dikonsumsi oleh masyarakat umum.

Deployment adalah proses pelatihan model dengan mengintegrasikannya ke dalam aplikasi atau sistem produksi, sehingga dapat digunakan oleh pengguna akhir (*end user*). Monitoring adalah proses untuk memastikan bahwa model bekerja dengan baik setelah deployment dan untuk mengidentifikasi jika ada degradasi kinerja atau masalah lain.





konsep Dasar Klasifikasi



Klasifikasi adalah teknik dalam machine learning untuk mengelompokkan data ke dalam kategori atau kelas



atau berdasarkan karakteristik atau fitur dari data tersebut. Klasifikasi termasuk dalam **supervised learning**, yaitu model dilatih menggunakan data yang sudah diberi label. Contoh aplikasi klasifikasi termasuk pengenalan wajah, filter spam pada email, deteksi penipuan, dan diagnosis penyakit.

Tujuan utama klasifikasi adalah memprediksi kelas atau label dari data baru yang belum pernah dilihat oleh model berdasarkan pola yang telah dipelajari dari data latih. Misalnya, jika kita melatih model dengan data buah-buahan yang memiliki fitur ukuran, warna, dan bentuk, model tersebut bisa digunakan untuk mengidentifikasi jenis buah baru sebagai apel, pisang, atau jeruk.

Proses Klasifikasi: Langkah Demi Langkah

Proses klasifikasi terdiri dari beberapa tahapan penting. Berikut langkah-langkahnya.

1. Pengumpulan Data

Data yang digunakan harus relevan, berkualitas baik, dan representatif terhadap masalah yang ingin diselesaikan. Data ini akan digunakan untuk melatih dan menguji model.

2. Pra-pemrosesan Data

Data mentah sering kali tidak siap digunakan oleh model. Pra-pemrosesan melibatkan membersihkan data, menangani data hilang, menghapus duplikasi, dan mengonversi data menjadi format yang bisa diproses oleh algoritma.

3. Pembagian Data

Data yang telah diproses kemudian dibagi menjadi dua set: data latih (training data) dan data uji (testing data). Biasanya, sekitar 70–80% digunakan untuk melatih model dan sisanya untuk menguji model.

4. Pemilihan Algoritma Klasifikasi

Pemilihan algoritma bergantung pada jenis data, ukuran dataset, dan kompleksitas masalah. Contoh algoritma klasifikasi yang populer adalah Decision Tree, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), dan Random Forest.

5. Pelatihan Model

Model dilatih menggunakan data latih. Pada tahap ini, algoritma akan mempelajari pola dari data dan membangun model yang bisa digunakan untuk memprediksi kelas dari data baru.

6. Evaluasi Model

Setelah model dilatih, kinerjanya dievaluasi menggunakan data uji. Metrik seperti **akurasi**, **precision**, **recall**, dan **F1-Score** digunakan untuk menilai performa model.





Jenis-Jenis Klasifikasi



Ada tiga jenis utama klasifikasi berdasarkan jumlah kelas atau label yang harus diprediksi.



Klasifikasi Biner

Mengelompokkan data menjadi dua kategori atau kelas, seperti "spam" dan "bukan spam."

2. Klasifikasi Multikelas

Melibatkan lebih dari dua kelas. Misalnya, model harus memprediksi bahwa sebuah gambar termasuk kategori "anjing," "kucing," atau "burung."

3. Klasifikasi Multilabel

Sebuah data bisa dikategorikan ke lebih dari satu kelas sekaligus. Misalnya, sebuah artikel berita bisa diklasifikasikan sebagai "teknologi" dan "bisnis" sekaligus.

Contoh Aplikasi Klasifikasi

- **Pengenalan Wajah:** Mengklasifikasikan gambar wajah berdasarkan identitas individu.
- **Deteksi Penipuan:** Mengidentifikasi sebuah transaksi keuangan tergolong penipuan atau tidak.
- **Filter Spam:** Mengklasifikasikan email sebagai spam atau tidak spam.

K-Nearest Neighbors (KNN)

Algoritma K-Nearest Neighbors (KNN) adalah metode supervised learning yang digunakan untuk klasifikasi dan regresi. Dikembangkan oleh Evelyn Fix dan Joseph Hodges pada tahun 1951, dan kemudian diperluas oleh Thomas Cover, KNN dikenal sebagai salah satu algoritma klasifikasi yang paling sederhana serta intuitif dalam machine learning.

Parameter Utama KNN

1. Jumlah Tetangga (K)

- **Definisi:** menentukan jumlah tetangga terdekat yang akan dipertimbangkan dalam klasifikasi.
- **Nilai K Kecil:** rentan terhadap *noise* dan *outlier*, berpotensi *overfitting*.
- **Nilai K Besar:** model lebih stabil, tetapi bisa *underfitting* karena pengaruh tetangga yang lebih jauh.

2. Metric Jarak

- **Euclidean Distance:** jarak garis lurus antara dua titik.
- **Manhattan Distance:** jarak berdasarkan jumlah perbedaan sepanjang sumbu koordinat.
- **Minkowski Distance:** generalisasi dari Euclidean dan Manhattan dengan parameter p.
- **Cosine Similarity:** mengukur kesamaan sudut antara dua vektor, sering digunakan untuk data teks.





4. Panjang Jarak (Distance Metric Parameters)

- **Contoh:** parameter p dalam Minkowski Distance yang memungkinkan penyesuaian metrik jarak.



Normalisasi Data

- **Standardisasi:** mengubah data sehingga memiliki mean 0 dan deviasi standar 1.
- **Normalisasi Min-Max:** mengubah data dalam rentang 0 hingga 1.

6. Algoritma Pencarian Tetangga

- **Brute Force:** menghitung jarak antara data baru dan semua titik dalam dataset.
- **KD-Tree:** struktur data untuk mempercepat pencarian tetangga terdekat.
- **Ball-Tree:** struktur data untuk data berdimensi tinggi.

Cara Kerja KNN

1. **Persiapan Data:** mengumpulkan dataset dengan fitur dan label.
2. **Pengukuran Jarak:** menghitung jarak antara data baru dan setiap data dalam dataset.
3. **Pemilihan Jumlah Tetangga (K):** menentukan berapa banyak tetangga yang akan dipertimbangkan.
4. **Identifikasi Tetangga Terdekat:** menentukan K tetangga terdekat dari data baru.
5. **Voting Mayoritas:** menentukan kelas berdasarkan mayoritas label dari tetangga terdekat.
6. **Pengambilan Keputusan Akhir:** memberikan prediksi akhir berdasarkan voting mayoritas.

Kelebihan dan Kekurangan KNN

• Kelebihan

- Sederhana dan intuitif.
- Non-parametrik dan cocok untuk berbagai jenis data.
- Efektif untuk dataset kecil.

• Kekurangan

- Komputasi berat untuk dataset besar.
- Sensitif terhadap *noise* dan data yang tidak relevan.
- Memori intensif karena menyimpan seluruh dataset.

Decision Tree

Decision Tree adalah algoritma machine learning yang digunakan dalam klasifikasi dan regresi. Struktur algoritma ini mirip dengan pohon dengan cabang yang mewakili keputusan berdasarkan fitur-fitur.

Parameter Utama Decision Tree





- **Log Loss:** alternatif lain untuk mengukur kualitas pembagian.



Maksimal Kedalaman (max_depth)

- **Kedalaman Terbatas:** membantu mencegah overfitting.
- **Kedalaman Tak Terbatas:** pohon tumbuh hingga mencapai batas alami.

Jumlah Minimum Sampel untuk Split (min_samples_split)

- **Nilai Kecil:** membagi node dengan sedikit sampel.
- **Nilai Besar:** memerlukan lebih banyak sampel untuk membagi node.

4. Jumlah Minimum Sampel untuk Daun (min_samples_leaf)

- **Nilai Kecil:** leaf node bisa berisi sedikit sampel.
- **Nilai Besar:** setiap leaf node harus berisi minimal sampel yang ditentukan.

5. Jumlah Maksimum Fitur untuk Pembagian (max_features)

- **Nilai Kecil:** hanya sebagian fitur yang dipertimbangkan untuk pembagian.
- **Nilai Besar:** semua fitur dipertimbangkan untuk pembagian.

6. Splitter

- **Best:** memilih pembagian terbaik berdasarkan kriteria.
- **Random:** memilih pembagian secara acak dari subset fitur.

Cara Kerja Decision Tree

1. **Pemisahan Data Awal:** memuat dataset dan membaginya berdasarkan fitur.
2. **Pemilihan Fitur dan Pembagian Data:** memilih fitur untuk membagi data menggunakan kriteria, seperti Gini atau Entropy.
3. **Pembangunan Pohon:** mengulangi proses pemilihan fitur dan pembagian hingga mencapai node daun.

Kelebihan dan Kekurangan Decision Tree

• Kelebihan

- Mudah dipahami dan diinterpretasikan.
- Memerlukan sedikit pra-pemrosesan data.

• Kekurangan

- Rentan terhadap overfitting.
- Pohon yang sangat dalam dapat menjadi kompleks dan sulit diinterpretasikan.

Random Forest





Cara Kerja Random Forest



Pengambilan Sampel Data

- Menggunakan teknik *bootstrap*, yaitu pengambilan sampel dengan penggantian dari dataset pelatihan asli.
- Setiap sampel data mungkin muncul lebih dari sekali dalam satu pohon.

2. Pembentukan Decision Trees

- Untuk setiap sampel data, Decision Tree dibangun dengan memilih subset acak dari fitur pada setiap node.

3. Proses Training

- Setiap Decision Tree dibangun hingga kedalaman maksimum yang ditentukan atau tidak ada pembagian lebih lanjut yang memungkinkan.

4. Agregasi Prediksi

- Untuk klasifikasi: Hasil akhir ditentukan berdasarkan voting mayoritas dari semua pohon.
- Untuk regresi: Hasil akhir adalah rata-rata dari prediksi semua pohon.

5. Evaluasi Model

- Diuji menggunakan data uji atau teknik validasi silang untuk mengevaluasi akurasi dan kemampuan generalisasi.

6. Penerapan Model

- Model yang telah dilatih digunakan untuk membuat prediksi pada data baru.

Parameter Utama Random Forest

1. **n_estimators**: jumlah Decision Trees dalam model. Semakin banyak pohon, semakin stabil model. Nilai default adalah 100, dan nilai yang disarankan berkisar antara 100 hingga 300.
2. **max_depth**: kedalaman maksimum setiap pohon. Menetapkan nilai ini dapat membantu mencegah *overfitting* dan *underfitting*. Nilai sering digunakan adalah antara 10–20.
3. **min_samples_split**: jumlah minimum sampel yang diperlukan untuk membagi node. Nilai default adalah 2. Nilai lebih tinggi membuat pohon lebih sederhana.
4. **min_samples_leaf**: jumlah minimum sampel yang harus ada pada leaf node. Nilai default adalah 1 dengan nilai antara 1 hingga 5 sering digunakan untuk dataset besar.
5. **max_features**: jumlah maksimum fitur yang dipertimbangkan untuk pemisahan setiap node. Nilai default adalah "sqrt" untuk klasifikasi dan "log2" untuk regresi.
6. **bootstrap**: menentukan bahwa sampel diambil dengan penggantian. Default adalah True yang meningkatkan variasi antar pohon.





Kelebihan dan Kekurangan Random Forest

Kelebihan



Akurasi tinggi.



Robust terhadap overfitting.



Kemampuan menangani data tidak seimbang.

- Menangani data hilang dengan baik.
- Menyediakan informasi tentang pentingnya fitur.

Kekurangan

- Kebutuhan memori yang tinggi.
- Interpretabilitas yang rendah.
- Lambat pada prediksi dengan banyak pohon.
- Kurang efektif pada data kecil.
- Waktu pelatihan yang lama.

Support Vector Machine (SVM)

Support Vector Machine (SVM) adalah algoritma machine learning yang digunakan terutama untuk klasifikasi. SVM mencari hyperplane yang optimal untuk memisahkan data ke dalam kelas berbeda dengan memaksimalkan margin antara hyperplane dan titik data terdekat dari setiap kelas.

Cara Kerja SVM

1. Pengumpulan dan Persiapan Data

Data dipersiapkan melalui pra-pemrosesan, termasuk pembersihan, penanganan nilai hilang, dan normalisasi.

2. Pemetaan Data ke Ruang Fitur yang Lebih Tinggi

Menggunakan fungsi kernel untuk memetakan data ke ruang fitur yang lebih tinggi, ini memungkinkan data non-linier untuk dipisahkan secara linier.

3. Menentukan Hyperplane

Mencari hyperplane yang memisahkan kelas dengan margin maksimum.

4. Identifikasi Support Vectors

Titik data yang terletak paling dekat dengan hyperplane, ini menentukan posisi dan orientasi hyperplane.

5. Penanganan Data Tidak Terpisahkan Secara Linier

Kernel trick digunakan untuk menangani data yang tidak dapat dipisahkan secara linier dalam ruang fitur asli.

6. Mengatur Parameter Regularization ©

Mengontrol kekuatan regularisasi dengan nilai C yang lebih tinggi mengurangi regularisasi dan nilai C yang lebih rendah meningkatkan regularisasi.





Model yang sudah dilatih diterapkan untuk prediksi pada data baru.

Parameter Utama SVM



Kernel: fungsi yang digunakan untuk mengubah data ke ruang fitur yang lebih tinggi. Tipe kernel termasuk linear, polynomial, RBF, sigmoid, dan precomputed.

Gamma: faktor yang menentukan seberapa besar pengaruh setiap titik data pada keputusan model. Nilai dapat diatur dengan 'scale', 'auto', atau nilai float.

3. **Regularization (C):** mengontrol kekuatan regularisasi untuk mencegah overfitting. Nilai yang lebih besar berarti kurang regularisasi dan nilai yang lebih kecil berarti lebih banyak regularisasi.
4. **Degree:** hanya berlaku untuk kernel polinomial, ini menentukan derajat polinomial yang digunakan untuk memetakan data ke ruang fitur yang lebih tinggi.

Kelebihan SVM

- **Margin Maksimal:** meningkatkan generalisasi.
- **Kemampuan Non-Linier:** menggunakan kernel trick untuk data non-linier.
- **Kinerja Baik pada Data Kecil:** efektif dengan dataset kecil.
- **Efektif pada Dimensi Tinggi:** berfungsi baik dengan banyak fitur.

Kekurangan SVM

- **Kompleksitas Komputasi:** lambat dan memerlukan banyak memori untuk dataset besar.
- **Pemilihan Parameter:** memerlukan tuning parameter yang hati-hati.
- **Tidak Biasa Menghasilkan Probabilitas:** tidak memberikan probabilitas klasifikasi.
- **Interpretasi Model Sulit:** sulit dipahami, terutama dengan kernel non-linier.

Naive Bayes

Naive Bayes adalah algoritma klasifikasi berbasis probabilitas yang didasarkan pada **Teorema Bayes**. Algoritma ini mengasumsikan bahwa fitur-fitur dalam data bersifat independen satu sama lain (asumsi "*naive*"), meskipun dalam kenyataannya, fitur-fitur seringkali saling terkait. Naive Bayes menghitung probabilitas suatu data termasuk dalam kelas tertentu berdasarkan **probabilitas prior** (kemunculan kelas secara umum) dan **probabilitas likelihood** (kemunculan fitur tertentu dalam kelas tersebut).

Cara Kerja Naive Bayes

1. **Mengumpulkan dan Menyiapkan Data:** mengumpulkan data yang diberi label, membersihkannya, dan mengidentifikasi fitur penting.
2. **Menghitung Probabilitas Prior:** menghitung probabilitas kemunculan setiap kelas tanpa mempertimbangkan



5. **Klasifikasi:** memilih kelas dengan probabilitas tertinggi sebagai hasil klasifikasi.

6. **Evaluasi dan Tuning:** menilai performa model dan melakukan penyempurnaan.



Implementasi Model: menggunakan model untuk prediksi pada data baru.

Metrik Utama

Laplace Smoothing: teknik untuk mencegah probabilitas nol dengan menambahkan nilai kecil pada fitur yang tidak muncul.

- **Probabilitas Prior:** probabilitas awal kemunculan kelas sebelum mempertimbangkan fitur.
- **Probabilitas Likelihood:** probabilitas kemunculan fitur tertentu dalam kelas.

Kelebihan Naive Bayes

- Mudah diimplementasikan dan sederhana secara matematis.
- Cepat dalam pelatihan dan prediksi, bahkan dengan dataset besar.
- Efisien untuk data dengan banyak fitur dan menangani data yang hilang.

Kekurangan Naive Bayes

- Asumsi independensi fitur sering tidak sesuai dengan kenyataan.
- Kinerja menurun pada data yang sangat tidak seimbang atau jika fitur saling bergantung.
- Terbatas pada distribusi data yang diasumsikan, misalnya Gaussian atau multinomial.

Naive Bayes cocok digunakan dalam aplikasi, seperti filter spam, analisis sentimen, dan pengenalan pola meskipun performanya bisa menurun jika asumsi model tidak terpenuhi.

Evaluasi Model Klasifikasi

Evaluasi model klasifikasi penting untuk menilai efektivitas model dalam memprediksi kategori yang benar. Berikut adalah metode dan metrik utama yang digunakan.

Confusion Matrix

Confusion matrix adalah tabel yang menunjukkan hasil prediksi model, ini memberikan pandangan rinci tentang kinerja model dalam berbagai kelas.

- **True Positives (TP):** kasus positif yang benar-benar diprediksi sebagai positif.
- **True Negatives (TN):** kasus negatif yang benar-benar diprediksi sebagai negatif.
- **False Positives (FP):** kasus negatif yang diprediksi sebagai positif (kesalahan tipe I).
- **False Negatives (FN):** kasus positif yang diprediksi sebagai negatif (kesalahan tipe II).

Contoh: Dalam model deteksi penyakit dengan 100 sampel:





- TN: 55 kasus negatif benar.
- FP: 5 kasus negatif salah.



rik Evaluasi Klasifikasi



Akurasi



- **Definisi:** Proporsi prediksi benar (positif dan negatif) terhadap total prediksi.
- **Contoh:** Jika dari 100 prediksi, 90 benar, akurasi adalah 90%. Akurasi dapat menyesatkan pada data tidak seimbang.

2. Precision

- **Definisi:** Rasio prediksi positif yang benar terhadap semua prediksi positif.
- **Contoh:** Dalam deteksi penipuan, precision tinggi berarti sebagian besar transaksi yang diklasifikasikan sebagai penipuan benar-benar penipuan.

3. Recall (Sensitivitas)

- **Definisi:** Rasio prediksi positif benar terhadap semua kasus positif yang sebenarnya.
- **Contoh:** Dalam skrining kanker, recall tinggi berarti model mendeteksi sebagian besar kasus kanker, ini mengurangi risiko terlewatnya diagnosis.

4. F1-Score

- **Definisi:** Rata-rata harmonis dari precision dan recall, ini memberikan gambaran tentang trade-off antara keduanya.
- **Contoh:** F1-Score berguna untuk situasi dengan ketidakseimbangan kelas, ini memberikan penilaian yang lebih seimbang antara precision dan recall.

Menggunakan kombinasi metrik, seperti akurasi, precision, recall, dan F1-Score memberikan gambaran yang lebih lengkap tentang kinerja model klasifikasi serta membantu untuk memahami kekuatan dan kelemahan model dalam menangani data serta membuat prediksi.



Pendahuluan Regresi dalam Konteks Machine Learning

Regresi adalah salah satu teknik dalam machine learning yang memiliki kesamaan dengan klasifikasi. Namun, perbedaan utama antara keduanya terletak pada hasil prediksinya. Pada klasifikasi, model machine learning tugas untuk memprediksi sebuah kelas atau kategori. Misalnya, apakah suatu email adalah spam atau bukan. Sebaliknya, pada regresi model akan **memprediksi sebuah nilai numerik yang bersifat kontinu**, seperti memprediksi harga rumah berdasarkan berbagai faktor seperti luas tanah, jumlah kamar, dan lokasi.

Regression Data

X ₁	X ₂	X ₃	X _p	Y
				5.2
				1.3
				23.0
				7.4

Target Numerik

Classification Data

X ₁	X ₂	X ₃	X _p	Y
				cat
				dog
				cat
				cat

Kategorikal "Labels"

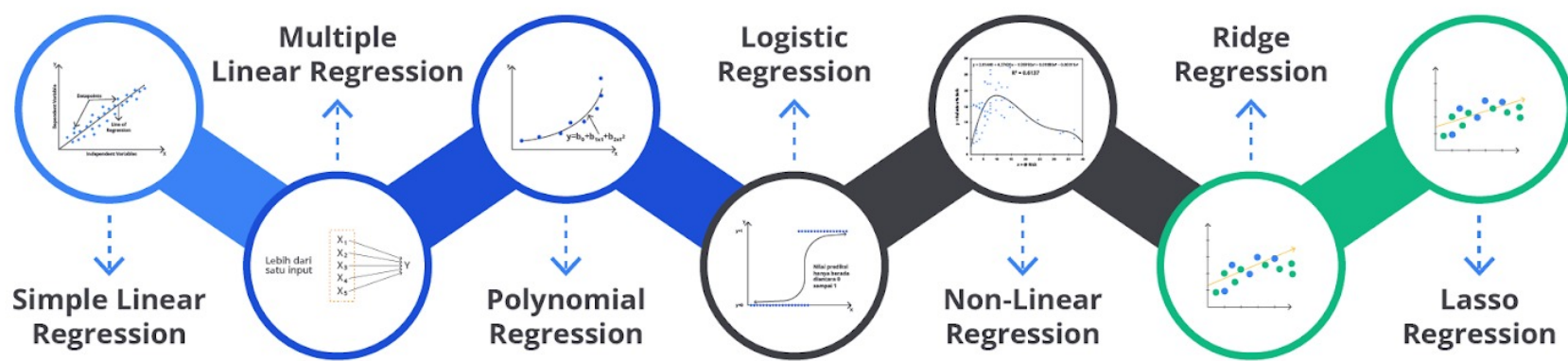
Bilangan kontinu yang diprediksi oleh model regresi adalah bilangan numerik yang tidak terbatas pada nilai-nilai diskrit. Sebagai contoh, dalam klasifikasi Anda mungkin memprediksi apakah seseorang lulus atau tidak lulus ujian (dua kelas atau kategori), sedangkan dalam regresi, Anda memprediksi nilai ujian seseorang dalam bentuk angka, misalnya 85,3.

Jenis-Jenis Regression

Seperti yang sudah Anda pelajari, regresi adalah salah satu teknik dalam statistik dan machine learning yang digunakan untuk memodelkan hubungan antara satu atau lebih variabel independen (input) dan variabel dependen (output).

Tujuan utama regresi adalah untuk memprediksi nilai dari variabel dependen berdasarkan nilai dari variabel independen. Namun, tahukah Anda bahwa setiap jenis regresi memiliki cara kerja dan kegunaan yang berbeda?

Berbagai Algoritma Regresi



Linear Regression

Linear regression (regresi linear) adalah jenis regresi yang paling sederhana, kita akan mencoba menemukan garis lurus terbaik yang menggambarkan hubungan antara variabel independen (X) dan variabel dependen (Y).

Misalnya, jika kita ingin memprediksi harga rumah berdasarkan ukuran rumah, kita bisa menggunakan regresi linear untuk menemukan garis yang paling cocok antara ukuran rumah (X) dan harga rumah (Y).

Multiple Linear Regression

Multiple Linear Regression (Regresi Linear Berganda) adalah pengembangan dari regresi linear sederhana yang digunakan untuk memodelkan hubungan antara satu variabel dependen (terkadang disebut variabel respons atau target) dan dua atau lebih variabel independen (juga disebut prediktor atau fitur). Model ini memungkinkan kita untuk memahami bagaimana beberapa faktor memengaruhi hasil yang diinginkan secara simultan.

Polynomial Regression

Polynomial Regression (regresi polinomial) adalah bentuk lanjutan dari regresi linear yang digunakan untuk memodelkan hubungan antara variabel independen dan variabel dependen ketika hubungan tersebut tidak linear. Sebagai pengembangan dari regresi linear, metode regresi polinomial memungkinkan hubungan antara variabel untuk berbentuk kurva dengan derajat yang lebih tinggi daripada garis lurus seperti parabola atau kurva lainnya.

Logistic Regression

Logistic Regression (regresi logistik) adalah salah satu teknik pemodelan statistik yang digunakan untuk memprediksi hasil biner, yaitu hasil dengan dua kemungkinan, seperti "ya" atau "tidak," "sukses" atau "gagal," dan lain sebagainya. Berbeda dengan regresi linear yang digunakan untuk memprediksi nilai numerik, regresi logistik digunakan untuk memodelkan probabilitas bahwa suatu kejadian akan terjadi (hasil biner).

linear. Dalam non-linear regression, model yang digunakan bisa berbentuk lebih kompleks, seperti eksponensial, logaritmik, kuadrat, kubik, atau bentuk fungsi lainnya.



? Ridge and Lasso Regression

Ⓜ Ridge Regression dan Lasso Regression adalah dua teknik regularisasi yang digunakan dalam regresi linear untuk mengatasi masalah multikolinearitas dan overfitting.

- **Ridge Regression**

Ridge regression menambahkan penalti berupa jumlah **kuadrat** dari koefisien regresi ke dalam fungsi loss.

Fungsi objektif yang diminimalkan dalam ridge regression dapat ditulis dengan rumus seperti berikut.

- **Lasso Regression**

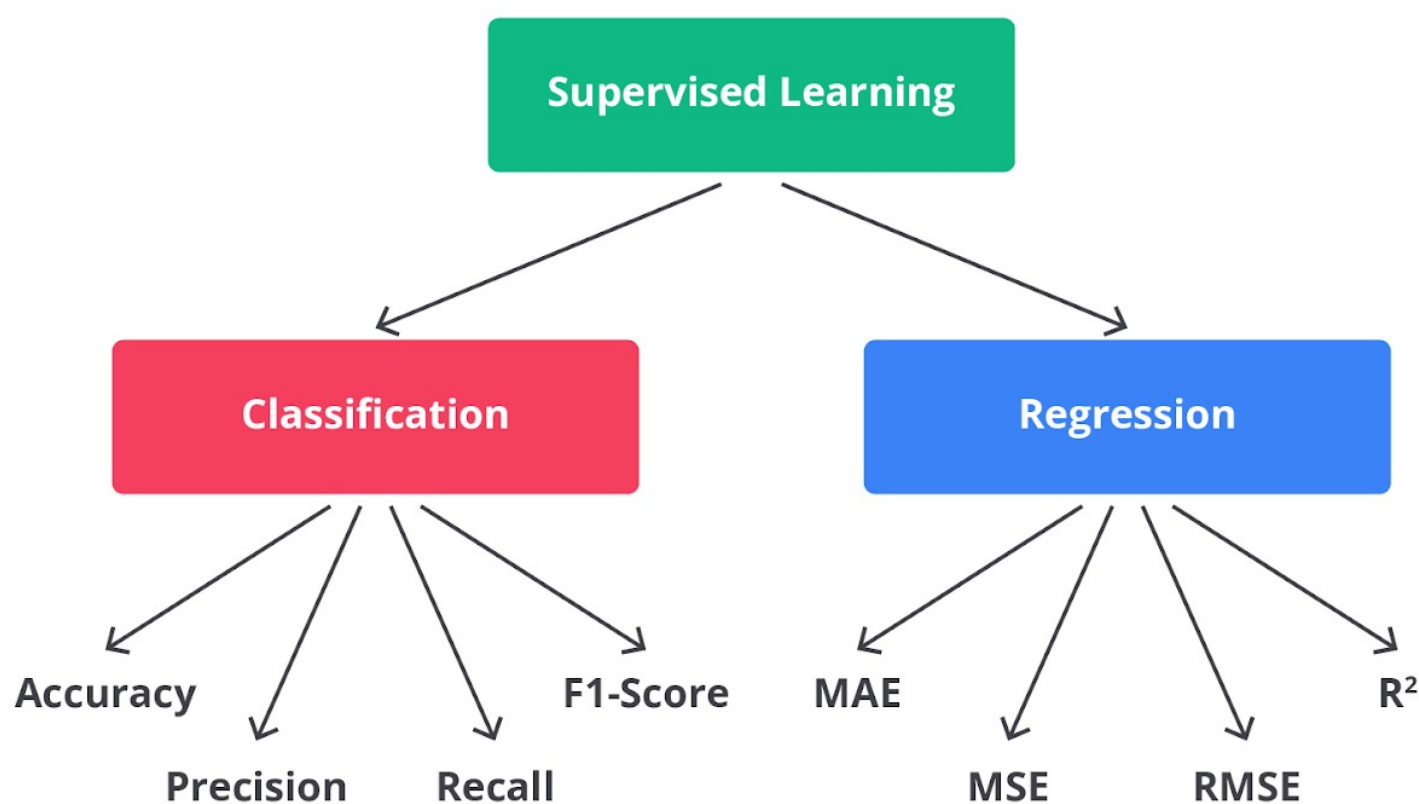
Lasso regression menambahkan penalti berupa jumlah **absolut** dari koefisien regresi ke dalam fungsi loss.

Fungsi objektif yang diminimalkan dalam lasso regression dapat ditulis dengan rumus seperti berikut.

Evaluasi Model Regresi

Evaluasi model regresi adalah langkah penting dalam proses membangun model prediksi. Dengan evaluasi yang baik, kita bisa mengetahui seberapa akurat model dalam memprediksi data yang belum pernah dilihat. Untuk membantu siswa pemula memahami proses ini, kami akan menjelaskan konsep dasar, metrik evaluasi utama, dan cara menginterpretasinya.

Setelah membangun model, penting untuk mengevaluasi seberapa baik model tersebut bekerja. Evaluasi ini membantu kita untuk memahami seberapa akurat prediksi model, mengetahui jika model terlalu rumit atau terlalu sederhana, dan memastikan bahwa model tidak hanya baik pada data latihan tetapi juga pada data baru.





MAE adalah rata-rata dari kesalahan dengan nilai absolut antara nilai sebenarnya dan nilai prediksi.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$



n Squared Error (MSE)

MSE adalah nilai rata-rata dari kuadrat kesalahan antara nilai sebenarnya dan nilai prediksi. MSE dapat digambarkan dengan rumus matematika seperti berikut.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE)

RMSE adalah akar kuadrat dari MSE. Ini mengembalikan kesalahan ke dalam satuan yang sama dengan data sehingga lebih mudah diinterpretasikan.

R-squared (R²)

R-squared juga dikenal sebagai coefficient of determination adalah salah satu metrik yang digunakan untuk mengevaluasi seberapa baik model regresi linear menjelaskan variasi dalam data. R-squared memberikan ukuran proporsi variasi dalam variabel dependen (output) yang dapat dijelaskan oleh variabel independen (input) dalam model.

R-squared dihitung dengan menggunakan perbandingan antara variasi total dalam data dan variasi yang dapat dijelaskan oleh model regresi. Secara matematis, R-squared dapat ditulis sebagai berikut.

$$R^2 = 1 - \frac{SSR}{SST}$$





Konsep Dasar dan Proses Clustering

Clustering adalah teknik unsupervised learning untuk mengelompokkan data ke dalam grup berdasarkan kemiripan fitur tanpa memerlukan label. Proses ini bermanfaat untuk menemukan pola atau struktur tersembunyi dalam data.

Penerapan dalam Proses Clustering

1. **Pengumpulan Data:** data relevan dikumpulkan untuk dianalisis, misalnya data pelanggan, seperti usia, pendapatan, dan riwayat pembelian.
2. **Pemilihan Fitur:** fitur penting dipilih berdasarkan relevansi dengan tujuan analisis, misalnya usia dan pendapatan.
3. **Pra-pemrosesan Data:** data dibersihkan, dihapus duplikasinya, dan dinormalisasi agar setiap fitur memiliki skala yang sama.
4. **Pemilihan Metode Pengukuran Jarak:** digunakan metode seperti jarak Euclidean untuk mengukur kesamaan antara objek data.
5. **Pemilihan Algoritma Clustering:** algoritma seperti K-Means, DBSCAN, atau hierarchical clustering dipilih berdasarkan kebutuhan analisis.
6. **Penerapan Algoritma Clustering:** algoritma diterapkan untuk membagi data ke dalam cluster berdasarkan kemiripan fitur.
7. **Evaluasi Hasil Clustering:** metrik seperti silhouette score digunakan untuk menilai kualitas cluster.
8. **Interpretasi dan Tindakan:** hasil clustering dianalisis untuk menemukan pola dan diambil tindakan, seperti perancangan strategi pemasaran berdasarkan segmentasi pelanggan.

Hierarchical Clustering (HC)

HC adalah teknik clustering yang membentuk struktur hierarki antara cluster. Ada dua pendekatan dalam HC:

- **Agglomerative** (bottom-up): setiap objek dimulai sebagai cluster terpisah dan digabungkan hingga menjadi satu cluster besar.
- **Divisive** (top-down): dimulai dengan satu cluster besar yang dibagi menjadi cluster lebih kecil.

Hierarchical clustering digambarkan dalam dendrogram untuk menunjukkan proses penggabungan atau pemecahan cluster.

Non-hierarchical Clustering (NHC)

Non-hierarchical Clustering (NHC) adalah metode clustering yang membagi data menjadi cluster tanpa



Metode Non-Hierarchical Clustering

Berikut adalah beberapa metode NHC yang sering digunakan:

K-Means Clustering



K-Means Clustering adalah metode yang membagi data menjadi kkk cluster berdasarkan jarak terdekat dari centroid (titik pusat cluster). Ini adalah salah satu metode yang paling umum digunakan karena kesederhanaannya dan efisiensinya.

Cara Kerja

1. **Inisialisasi:** Pilih jumlah cluster kkk dan inisialisasi centroid secara acak atau menggunakan metode seperti K-Means++.
2. **Penugasan Cluster:** Alokasikan setiap titik data ke centroid terdekat berdasarkan jarak Euclidean.
3. **Pembaruan Centroid:** Hitung ulang centroid sebagai rata-rata titik data dalam cluster.
4. **Iterasi:** Ulangi langkah penugasan dan pembaruan hingga centroid stabil atau konvergen.

Kelebihan

- **Simplicity:** mudah diimplementasikan dan dipahami.
- **Efisiensi:** cepat dalam komputasi, terutama untuk dataset besar.

Kekurangan

- **Pemilihan Jumlah Cluster:** membutuhkan penentuan jumlah cluster kkk sebelumnya.
- **Sensitivitas terhadap Outliers:** dapat terpengaruh oleh outliers yang memengaruhi posisi centroid.

K-Medoids Clustering

K-Medoids Clustering adalah varian dari K-Means yang menggunakan titik data nyata sebagai pusat cluster (medoids), bukan rata-rata. Ini membuatnya lebih robust terhadap noise dan outliers.

Cara Kerja

1. **Inisialisasi:** Tentukan jumlah cluster kkk dan pilih medoids secara acak.
2. **Penugasan Cluster:** Alokasikan setiap titik data ke medoid terdekat.
3. **Pemilihan Medoid Baru:** Pilih medoid baru untuk meminimalkan total jarak dalam cluster.
4. **Iterasi:** Ulangi langkah penugasan dan pemilihan medoid hingga tidak ada perubahan signifikan.

Kelebihan

- **Robust terhadap Outliers:** lebih tahan terhadap outliers.
- **Kualitas Cluster:** medoids memberikan pusat yang lebih representatif.



DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN adalah metode clustering berbasis kepadatan yang mengidentifikasi cluster berdasarkan kepadatan titik. Tidak memerlukan jumlah cluster yang ditentukan sebelumnya dan dapat menemukan cluster dengan bentuk arbitrer.

Cara Kerja

1. **Penentuan Parameter:** Tentukan parameter jarak maksimum (ϵ) dan jumlah minimum titik (MinPts).
2. **Pengelompokan:** Identifikasi titik-titik yang cukup dekat satu sama lain untuk membentuk cluster. Titik yang tidak memenuhi kriteria dianggap sebagai noise.
3. **Ekspansi Cluster:** Tambahkan titik tetangga yang memenuhi kriteria kepadatan.

Kelebihan

- **Tidak Memerlukan Jumlah Cluster:** tidak perlu menentukan jumlah cluster sebelumnya.
- **Menangani Noise:** dapat mengidentifikasi noise dan cluster dengan bentuk arbitrer.

Kekurangan

- **Pemilihan Parameter:** memerlukan pemilihan parameter yang tepat (ϵ dan MinPts).
- **Skalabilitas:** kurang efisien pada dataset yang sangat besar.

Mean Shift Clustering

Mean Shift Clustering adalah metode berbasis kepadatan yang mencari titik-titik yang berkumpul di sekitar jendela pencarian (kernel) tanpa memerlukan jumlah cluster awal.

Cara Kerja

1. **Inisialisasi:** Tentukan ukuran jendela pencarian atau kernel.
2. **Pindah Titik:** Hitung rata-rata posisi titik di sekitar jendela dan geser titik ke posisi rata-rata tersebut.
3. **Konsolidasi:** Titik-titik yang berpindah ke posisi yang sama dikelompokkan bersama untuk membentuk cluster.

Kelebihan

- **Tidak Memerlukan Jumlah Cluster:** tidak perlu menentukan jumlah cluster sebelumnya.
- **Menangani Bentuk Arbitrer:** efektif dalam menemukan cluster dengan bentuk arbitrer.

Kekurangan



← Belajar Machine Learning untuk Pemula



Cari modul/konten

CTRL /





Pendahuluan Feature Engineering

Feature engineering adalah proses yang sangat penting dalam *machine learning*, di mana data mentah atau kotor akan menjadi fitur-fitur yang dapat digunakan oleh model untuk melakukan prediksi. Proses ini melibatkan pemilihan, transformasi, dan penciptaan fitur-fitur yang relevan serta bermakna dari data yang tersedia. Tujuan utamanya adalah untuk memaksimalkan performa model dengan menyediakan input yang paling representatif dan informatif.

Dengan melakukan feature engineering Anda dapat menciptakan, mengubah, dan memilih fitur yang relevan sehingga dapat membantu model memahami pola dalam data dengan lebih baik, meningkatkan akurasi prediksi, dan memastikan bahwa model dapat digeneralisasikan dengan baik pada data baru. Meskipun banyak model modern memiliki kemampuan untuk mengekstraksi fitur secara otomatis, *feature engineering* tetap memberikan dampak yang signifikan terutama ketika pengetahuan domain dapat diterapkan dengan baik.

Dengan memproses dan mentransformasi data, Anda dapat mengurangi *noise* yang mungkin mengaburkan pola penting dan meningkatkan hubungan (korelasi) yang benar-benar berharga antara setiap variabel. Proses ini juga membantu menyesuaikan data dengan algoritma yang akan digunakan sehingga model dapat bekerja lebih optimal dengan data yang telah diolah.

Mengurangi risiko *overfitting* juga merupakan salah satu manfaat utama dari *feature engineering*. Dengan melakukan pemilihan fitur yang tepat dan eliminasi fitur yang tidak relevan, dapat mencegah model menjadi terlalu kompleks. Hal ini dapat membantu dalam mengoptimalkan kinerja model agar lebih efisien sehingga memungkinkan model mencapai performa yang sangat baik dengan fitur-fitur yang tepat tanpa memerlukan komputasi yang besar (efisiensi).

Teknik Pemilihan Fitur (*Feature Selection*)

Teknik Pemilihan Fitur (Feature Selection) adalah proses pemilihan subset fitur yang paling relevan atau penting untuk digunakan dalam model machine learning. Tujuannya adalah untuk meningkatkan performa model dengan menghilangkan fitur yang tidak relevan, redundan, atau kotor (biasa disebut *noise*). Dengan demikian, kita bisa mengurangi kompleksitas model, meningkatkan akurasi, dan mempercepat proses pelatihan.

Pemilihan fitur sangat penting karena model machine learning yang dilatih dengan fitur yang tidak relevan atau terlalu banyak fitur, memungkinkan untuk mengalami masalah seperti *overfitting* yaitu kondisi ketika model terlalu cocok dengan data pelatihan sehingga tidak dapat melakukan generalisasi dengan baik untuk data baru. Dengan melakukan feature selection, kita dapat menghasilkan model yang lebih simpel, cepat, dan efektif.

Encoding Variabel ke Numerik

Encoding dalam machine learning adalah proses mengonversi data non-numerik (kategorikal atau teks) menjadi



7 Teknik Encoding

1) One-hot Encoding

Warna	Hijau	Merah	Hitam	Oranye
Hijau	1	0	0	0
Merah	0	1	0	0
Hitam	0	0	1	0
Oranye	0	0	0	1

2) Dummy Encoding

Hijau	Merah	Hitam	Oranye
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

3) Effect Encoding

Hijau	Merah	Hitam
1	0	0
0	1	0
0	0	1
-1	-1	-1

Hapus satu fitur secara random setelah melakukan one-hot encoding

Ganti nilai pada fitur yang dihilangkan pada dummy encoding menjadi -1

4) Label Encoding

Warna	Encoding
Hijau	1
Merah	2
Hitam	3
Hijau	1

Ubah setiap label unik menjadi angka

5) Ordinal Encoding

Ukuran	Encoding
XS	1
S	2
L	3
M	4

Ubah setiap label unik menjadi angka dengan urutan yang sudah ditentukan

6) Count Encoding

Warna	Encoding
Hijau	2
Merah	1
Hitam	1
Hijau	2

Encode berdasarkan jumlah frekuensi label

7) Binary Encoding

Warna	Warna_0	Warna_1
Green	0	0
Red	0	1
Black	1	0
Green	1	1

Ubah label menjadi nilai biner

Tujuan utamanya agar data tersebut dapat dimengerti oleh komputer sehingga dapat menjadi model yang dapat digunakan. Hal ini sangat penting karena sebagian besar algoritma machine learning bekerja dengan angka untuk menghitung jarak, perhitungan statistik, dan pengambilan keputusan.

Binning Numerik ke Kategorikal

Binning digunakan untuk mengubah data numerik kontinu menjadi kategori atau interval diskrit. Tujuannya adalah untuk menyederhanakan data numerik dengan memisahnya menjadi beberapa kelompok atau bin berdasarkan rentang atau distribusi nilai tertentu.

Feature Engineering

Kategorikal menjadi kontinu

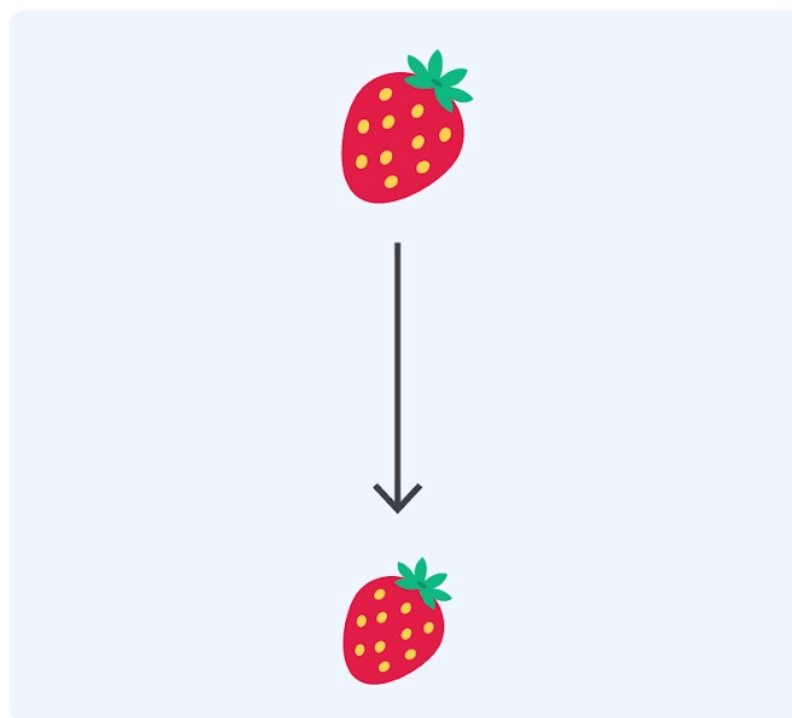
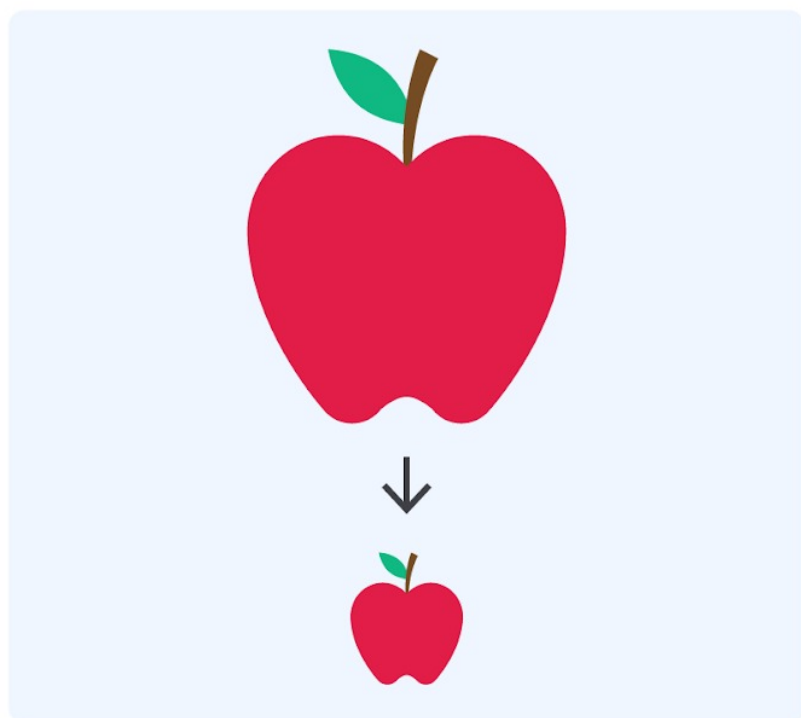
Kontinu menjadi kategorikal

Feature EncodingLabel
EncodingBinary
EncodingOrdinal
EncodingOne Hot
EncodingFrequency
EncodingDummy
Encoding**Feature Binning**Unsupervised
BinningSupervised
BinningEqual Width
BinningFrequency
Width BinningEntropy
Based Binning

Scaling Fitur

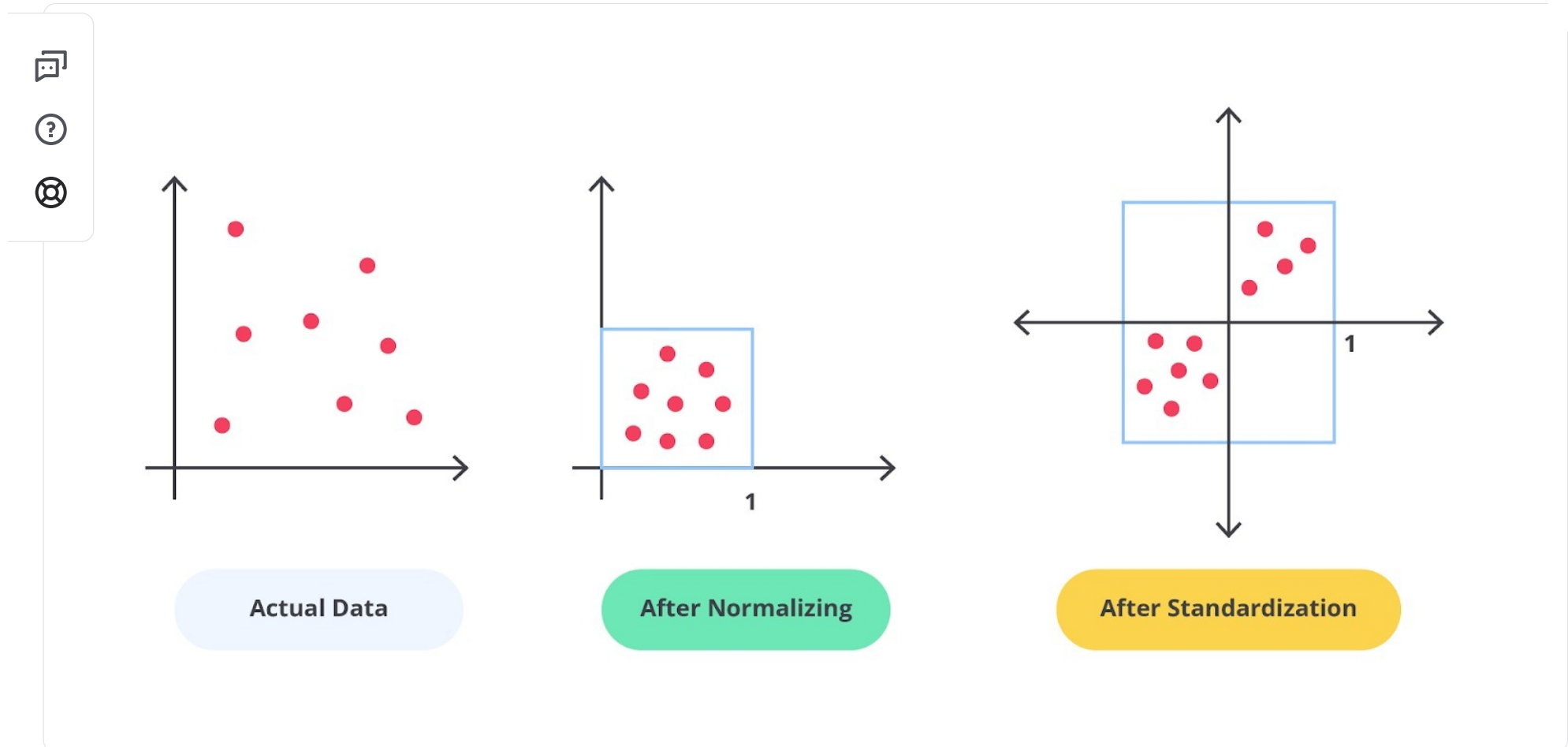
Scaling feature pada machine learning adalah proses menyesuaikan rentang atau skala nilai-nilai fitur agar berada dalam rentang tertentu yang lebih seragam. Proses ini memiliki peran yang cukup penting karena banyak algoritma machine learning sensitif terhadap perbedaan skala antara fitur.

Scaling



Jika nilai pada masing-masing fitur tidak di-*scaling*, algoritma yang Anda gunakan mungkin akan memberikan bobot

Secara umum terdapat dua teknik yang dapat Anda gunakan untuk mengubah skala data agar konsisten yaitu normalisasi dan standardisasi. Perhatikan perbedaan pada gambar berikut.



Penanganan Outlier

Dalam statistik, outlier adalah sebuah nilai yang jauh berbeda dari kumpulan nilai lainnya dan dapat mengacaukan hasil dari sebuah analisis statistik. Outlier dapat disebabkan oleh kesalahan dalam pengumpulan data atau nilai tersebut benar ada dan memang unik dari kumpulan nilai lainnya.

Apa pun alasan kemunculannya, Anda perlu tahu cara mengidentifikasi dan memproses outlier. Ini adalah bagian penting dalam persiapan data di dalam machine learning. Salah satu cara termudah untuk mengecek apakah terdapat outlier dalam data kita adalah dengan melakukan visualisasi.

Outliers



Dapat dilihat dengan jelas bahwa terdapat satu sampel yang jauh berbeda dengan sampel-sampel lainnya. Setelah mengetahui bahwa di data kita terdapat outlier, kita dapat mencari lalu menghapus sampel tersebut dari dataset.

Oversampling dan Undersampling

Imbalanced Dataset adalah situasi ketika jumlah data pada satu kelas jauh lebih sedikit atau lebih banyak dibandingkan dengan kelas lainnya. Ini sering terjadi dalam masalah klasifikasi, khususnya dalam binary classification (klasifikasi dua kelas) ketika satu kelas (sering disebut minoritas) memiliki jauh lebih sedikit sampel dibandingkan kelas lainnya (disebut mayoritas). Namun, masalah ini juga tidak menutup kemungkinan terjadi pada masalah klasifikasi lebih dari dua kelas atau multi kelas, ya.

Ketika dataset tidak seimbang, algoritma machine learning cenderung lebih baik dalam memprediksi kelas mayoritas dan mengabaikan atau salah mengklasifikasikan kelas minoritas. Ini terjadi karena sebagian besar algoritma secara alami berusaha memaksimalkan akurasi yang mungkin bukan metrik yang tepat untuk dataset yang tidak seimbang.

Misalnya, jika 95% dari data termasuk dalam kelas mayoritas dan hanya 5% dalam kelas minoritas, model yang selalu memprediksi kelas mayoritas dapat mencapai akurasi 95% tanpa pernah memprediksi kelas minoritas dengan benar.

Google secara resmi memberikan tiga buah level untuk kondisi imbalance yang berbeda-beda dihitung dari proporsi ketidakseimbangannya.

Percentage of data belonging to minority class	Degree of imbalance
20-40% of the dataset	Mild
1-20% of the dataset	Moderate

membandingkan hasil dari dataset yang imbalance dengan eksperimen kedepannya. Setelah itu, Anda perlu mencoba beberapa teknik untuk mengatasi permasalahan imbalance dataset seperti *oversampling* atau *undersampling*.



rsampling

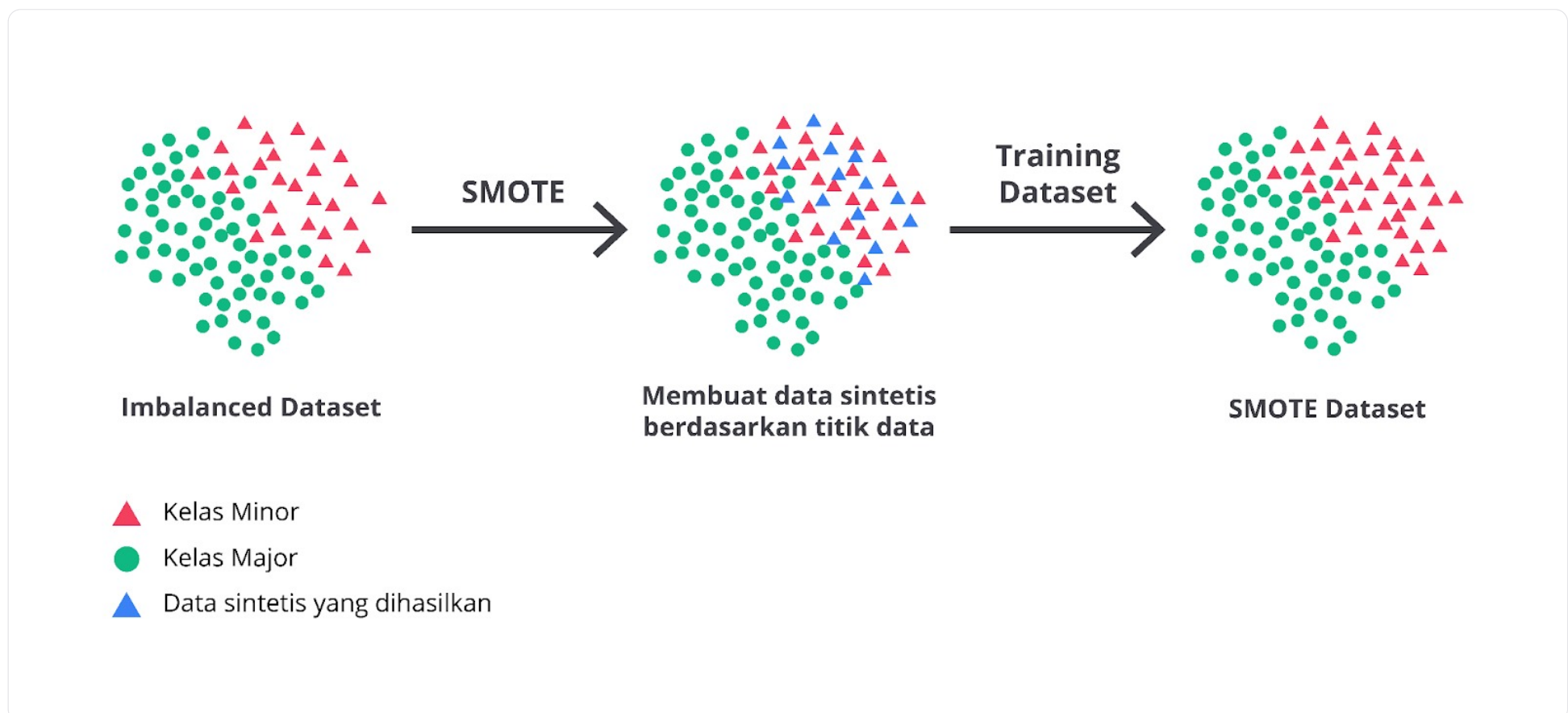
rsampling adalah metode yang menambahkan sampel pada kelas minoritas sehingga jumlahnya menjadi seimbang dengan kelas mayoritas. Teknik ini bekerja dengan cara memperbanyak data dari kelas minoritas agar model memiliki kesempatan yang lebih baik untuk belajar dari data tersebut.

Undersampling

Berbanding terbalik dengan oversampling, undersampling adalah metode yang mengurangi jumlah sampel dari kelas mayoritas agar sesuai dengan jumlah sampel di kelas minoritas. Teknik ini bekerja dengan menghapus sebagian data dari kelas mayoritas untuk menciptakan keseimbangan antara kelas-kelas.

SMOTE (*Synthetic Minority Over-Sampling Technique*)

Synthetic Minority Over-sampling Technique (SMOTE) adalah teknik oversampling yang dirancang untuk menangani masalah imbalanced dataset pada machine learning. Berbeda dengan random oversampling yang hanya menduplikasi sampel dari kelas minoritas, SMOTE menghasilkan sampel sintetis (baru) berdasarkan data yang sudah ada di kelas minoritas. Dari hal itu terciptalah variasi baru di dalam dataset dan mengurangi risiko overfitting yang mungkin terjadi jika kita hanya menduplikasi data secara acak.



Sebenarnya metode SMOTE juga bekerja dengan cara yang tidak terlalu rumit. Secara umum, metode ini memiliki tiga buah langkah yang harus dilakukan hingga menghasilkan data yang seimbang.

1. **Ambil Sampel dari Kelas Minoritas:** setiap sampel minoritas yang ada akan memilih beberapa tetangga



menghubungkan dua data minoritas.

3. **Tambahkan Sampel Sintetis ke Dataset:** sampel baru ini ditambahkan ke dataset, sehingga jumlah data di kelas minoritas bertambah hingga proporsional dengan kelas mayoritas.



Anda lebih memahami proses yang terjadi pada teknik ini, mari kita lakukan perhitungan sederhana menggunakan teknik SMOTE.





biasa!

Modul ini berfokus pada dua tantangan utama dalam machine learning, yaitu overfitting dan underfitting.

Ini akan banyak memengaruhi kemampuan model untuk membuat prediksi secara akurat pada data yang belum pernah dilihat sebelumnya. Pemahaman serta penanganannya sangat penting untuk mengembangkan model yang efektif.

Overfitting terjadi ketika model terlalu kompleks sehingga mampu mempelajari data latih dengan sangat baik, termasuk *noise* atau fluktuasi acak. Model seperti ini akan memiliki performa tinggi pada data latih, tetapi cenderung memiliki kinerja buruk pada data baru. Sebaliknya, **underfitting** terjadi ketika model terlalu sederhana untuk menangkap pola mendalam dari data sehingga tidak dapat memberikan prediksi yang baik pada data latih maupun data baru. Ini biasanya disebabkan oleh model yang tidak cukup kompleks atau fitur tidak memadai.

Modul ini mengajarkan Anda cara menyeimbangkan kompleksitas model dengan kualitas data untuk mencapai *good fit*—keadaan ideal saat model memiliki generalisasi yang baik tanpa overfitting atau underfitting. Dengan menerapkan teknik-teknik ini secara bijaksana, Anda dapat mengembangkan model machine learning yang lebih robust dan dapat diandalkan. Ini memastikan performa baik tidak hanya pada data latih, tetapi juga pada data baru yang dihadapi dalam aplikasi nyata.

Ingatlah bahwa keseimbangan adalah kunci. Teruslah bereksperimen dan belajar karena setiap dataset serta masalah dapat memerlukan pendekatan yang berbeda. Selamat mencoba, semoga model-model Anda semakin tajam dan andal, ya!

Berikut adalah rangkuman dari Modul 7: Overfitting dan Underfitting.

Dalam machine learning, tujuan utama adalah membangun model yang dapat menggeneralisasi dengan baik dari data latih untuk membuat prediksi akurat pada data baru. Keseimbangan antara belajar pola penting dan menghindari pemahaman berlebihan adalah kunci serta masalah umum yang muncul adalah overfitting dan underfitting.

Overfitting

- **Definisi:** Ini terjadi ketika model belajar terlalu banyak dari data latih, termasuk detail dan *noise* yang tidak relevan.
- **Gejala:** Model menunjukkan performa sangat baik pada data latih, tetapi buruk dalam data uji.
- **Analogi:** Ini seperti menghafal setiap soal dari buku latihan tanpa memahami konsep sehingga kesulitan menghadapi variasi soal ujian.
- **Contoh:** Model prediksi harga rumah yang menghafal harga spesifik dari data latih termasuk outlier sehingga kesulitan dengan data baru.

Underfitting

- **Definisi:** Ini terjadi ketika model terlalu sederhana dan tidak mampu menangkap pola signifikan dalam data.
- **Gejala:** Model memiliki performa buruk, baik pada data latih maupun data uji.





Good Fit



Definisi: Kondisi ideal ketika model dapat menangkap pola signifikan tanpa terlalu terikat pada detail yang tidak relevan atau terlalu sederhana.



Gejala: Model memberikan prediksi yang akurat pada data latih dan data uji.



Analogi: Ini seperti mempersiapkan ujian dengan memahami buku secara menyeluruh dan berbagai jenis soal sehingga mampu menjawab dengan baik.

- **Contoh:** Model prediksi harga rumah yang memahami hubungan kompleks antara berbagai fitur, seperti lokasi, ukuran, dan kondisi.

Penyebab Overfitting dan Underfitting

- **Kompleksitas Model yang Terlalu Tinggi:** Model dengan terlalu banyak parameter atau fitur dapat menangkap *noise* dan menyebabkan overfitting.
- **Data Latih yang Terbatas:** Model terlalu menyesuaikan dengan data latih yang sedikit dan mengakibatkan overfitting.
- **Pelatihan yang Terlalu Lama:** Model yang dilatih terlalu lama dapat mulai mempelajari pola tidak relevan.
- **Fitur yang Tidak Relevan:** Penggunaan fitur yang tidak penting membuat model terlalu kompleks dan cenderung overfit.

Memahami dan mengatasi penyebab ini adalah kunci untuk mengembangkan model yang efektif serta dapat diandalkan.

Metode Deteksi Overfitting

1. Evaluasi Performa pada Data Latih dan Data Uji

- Jika model berperforma sangat baik pada data latih, tetapi buruk dalam data uji, ini menunjukkan overfitting. Model mungkin "mengingat" data latih secara berlebihan dan gagal menggeneralisasi.

2. Learning Curve

- Learning curve yang menunjukkan kesalahan rendah pada data latih, tetapi tinggi dalam data uji dapat menandakan overfitting. Kurva ini memvisualisasikan bahwa model terlalu kompleks.

Metode Deteksi Underfitting

1. Evaluasi Performa pada Data Latih dan Data Uji

- Model menunjukkan performa buruk, baik pada data latih maupun data uji yang mengalami underfitting. Model mungkin terlalu sederhana untuk menangkap pola kompleks dalam data.

2. Learning Curve

- Learning curve dengan kesalahan tinggi pada kedua set data yang tidak menurun menunjukkan





perlu penyesuaian untuk meningkatkan kompleksitas model.

Teknik Mengatasi Overfitting



Cross-Validation

- Membagi data menjadi beberapa subset untuk memastikan model tidak overfitting pada subset tertentu dan dapat menggeneralisasi dengan baik.

2. Early Stopping

- Menghentikan pelatihan saat model mulai overfit pada data validasi meskipun performa dalam data latih masih meningkat.

3. Regularization

- **L1 Regularization (Lasso):** Menyederhanakan model dengan mengurangi koefisien fitur yang kurang penting menjadi nol.
- **L2 Regularization (Ridge):** Mengurangi ukuran koefisien fitur yang besar tanpa menghilangkan fitur sama sekali.
- **Elastic Net:** Kombinasi L1 dan L2 untuk memilih fitur penting tanpa kehilangan informasi dari fitur terkait.

4. Dropout

- Menghilangkan neuron secara acak selama pelatihan untuk mencegah model terlalu bergantung pada neuron tertentu.

5. Data Augmentation

- Meningkatkan variasi data latih untuk membantu model lebih robust terhadap berbagai situasi.

6. Pruning

- Menyederhanakan pohon keputusan dengan menghilangkan cabang yang tidak signifikan untuk mengurangi kompleksitas dan risiko overfitting.

Teknik Mengatasi Underfitting

1. Gunakan Model yang Lebih Kompleks

- Beralih ke model lebih kompleks, seperti Decision Trees, Random Forests, atau neural networks untuk menangkap hubungan yang lebih rumit dalam data.

2. Tambahkan Data Latih

- Menambah jumlah data latih untuk memberikan model lebih banyak variasi dan contoh untuk belajar serta membantu model dalam menangkap pola yang lebih kompleks.

3. Perbaiki Preprocessing Data





banyak dan relevan.

Eksperimen dengan Hyperparameter Tuning



- Mengatur parameter model untuk menemukan konfigurasi optimal yang dapat meningkatkan kinerja model.

Dengan mengidentifikasi dan mengatasi overfitting serta underfitting menggunakan teknik-teknik ini, Anda dapat membangun model machine learning secara lebih akurat dan efektif. Ini dapat menggeneralisasi dengan baik pada data yang belum pernah dilihat sebelumnya.





Pendahuluan Hyperparameter Tuning



Hyperparameter tuning adalah proses dalam machine learning untuk mengoptimalkan kinerja model berdasarkan nilai tertentu. Singkatnya, parameter model adalah nilai yang dipelajari oleh model selama proses pelatihan, seperti bobot (*weight*) dalam jaringan saraf tiruan. Di lain sisi, hyperparameter merupakan nilai yang tidak dipelajari selama pelatihan, tetapi dapat ditentukan sebelum pelatihan model dimulai.

Strategi Hyperparameter Tuning

Dari sekian banyak hyperparameter tentunya tidak semuanya memiliki dampak yang sama pada model tertentu sehingga penting untuk memilih dan mengatur hyperparameter yang relevan untuk mencapai performa optimal.

Berikut adalah panduan umum tentang pemilihan hyperparameter yang relevan, termasuk cara untuk mengidentifikasi dan mengatur prioritas dalam tuning.

1. Penggunaan Default Hyperparameter

Pada tahap awal, Anda dapat memulai dengan nilai *default* dari hyperparameter yang sering kali sudah diatur sedemikian rupa oleh library yang digunakan agar memberikan hasil yang baik pada kebanyakan kasus. Nilai ini dapat memberikan *baseline* yang cukup baik sebelum mulai melakukan tuning secara manual maupun otomatis.

2. Memahami Algoritma yang Digunakan

Setiap algoritma machine learning memiliki hyperparameter yang berbeda-beda dan dapat memengaruhi model machine learning. Oleh karena itu, langkah pertama dalam pemilihan hyperparameter yang relevan adalah memahami algoritma yang digunakan, termasuk apa saja hyperparameter penting yang tersedia.

3. Identifikasi Hyperparameter yang Paling Berpengaruh

Salah satu cara terbaik untuk mengidentifikasi hyperparameter yang relevan adalah sebagai berikut.

- **Riset Literatur dan Dokumentasi:** memahami dari dokumentasi atau penelitian yang relevan untuk mencari hyperparameter terpenting pada algoritma yang digunakan.
- **Pengalaman adalah Guru Terbaik:** pengalaman mengembangkan model sering kali memberikan wawasan tentang hyperparameter mana yang layak untuk difokuskan.
- **Eksperimen Awal:** lakukan eksperimen awal dengan nilai default untuk semua hyperparameter, kemudian lakukan analisis untuk memahami hyperparameter mana yang paling memengaruhi kinerja.

4. Prioritaskan Tuning Hyperparameter yang Krusial

Setelah mengidentifikasi hyperparameter yang relevan, langkah selanjutnya adalah memprioritaskan tuning hyperparameter yang paling krusial. Ini berarti, Anda sebaiknya tidak mencoba menyetel semua hyperparameter sekaligus, tetapi memfokuskan eksplorasi pada hyperparameter yang berpotensi





tergantung pada pengaturan hyperparameter lainnya. Misalnya, dalam jaringan saraf tiruan, learning rate dan momentum sering kali berinteraksi bersamaan.



Menyesuaikan Hyperparameter Berdasarkan Data



Hyperparameter yang relevan tidak hanya bergantung pada algoritma yang digunakan tetapi juga pada



karakteristik data yang dilatih. Beberapa pertimbangan yang biasanya dilakukan yaitu seperti berikut.

- **Ukuran Dataset:** pada dataset besar, batch size yang lebih besar bisa membuat pelatihan lebih efisien. Sementara pada dataset kecil, batch size yang kecil mungkin lebih tepat untuk mencegah overfitting.
- **Dimensi Data:** pada dataset berdimensi tinggi, model seperti SVM sering kali membutuhkan tuning yang lebih intensif pada hyperparameter gamma untuk menangani kompleksitas data.
- **Jumlah Noise:** jika data mengandung banyak noise, regularization parameter misalnya C pada SVM atau alpha pada Ridge Regression mungkin harus diperhatikan supaya terhindar dari overfitting.

7. Evaluasi Kinerja Model dengan Cross-Validation

Untuk setiap kombinasi hyperparameter yang diuji, Anda perlu melakukan evaluasi kinerja model secara menyeluruh (tidak hanya satu kali pengujian). Penggunaan teknik seperti *cross-validation* akan sangat membantu karena memungkinkan mengevaluasi performa model pada berbagai subset data dalam satu waktu sehingga hasil tuning hyperparameter dapat divalidasi dengan lebih cermat.

Grid Search

Grid Search adalah salah satu metode hyperparameter tuning yang digunakan untuk menemukan kombinasi hyperparameter optimal pada model machine learning. Grid Search bekerja dengan mencoba semua kombinasi dari nilai hyperparameter yang telah Anda tentukan dan mengevaluasi performa model untuk setiap kombinasi tersebut.

Tujuan dari Grid Search adalah untuk mengidentifikasi set hyperparameter yang menghasilkan performa terbaik berdasarkan metrik evaluasi yang dipilih (misalnya akurasi, F1-score, atau MSE).

Random Search

Random Search adalah salah satu metode hyperparameter tuning yang cenderung lebih efisien dibandingkan Grid Search. Alih-alih mencoba semua kombinasi hyperparameter seperti dalam Grid Search, Random Search memilih beberapa kombinasi hyperparameter secara acak dari ruang pencarian yang sudah ditentukan. Proses ini memungkinkan model untuk diuji dengan kombinasi acak yang dipilih secara independen untuk setiap iterasi.

Tujuan Random Search adalah menghemat waktu dan sumber daya dengan tetap melatih sejumlah kombinasi yang cukup representatif dari ruang pencarian, tanpa perlu menguji semua kombinasi yang mungkin terjadi.





Bayesian Optimization adalah salah satu teknik hyperparameter tuning yang efisien dan kompleks. Metode ini digunakan untuk menemukan kombinasi hyperparameter yang optimal dengan melakukan lebih sedikit percobaan dibandingkan Grid Search atau Random Search.



Bayesian Optimization sangat berguna untuk masalah yang memerlukan tuning hyperparameter pada ruang pencarian yang besar, di mana mencoba semua kombinasi hyperparameter akan sangat tidak efisien dan memakan waktu.

Bayesian Optimization tidak melakukan pencarian secara acak atau mencoba semua kombinasi, melainkan menggunakan pendekatan probabilistik untuk secara cermat memilih kombinasi hyperparameter yang paling mungkin memberikan hasil terbaik berdasarkan percobaan sebelumnya. Dengan demikian, metode ini sangat efektif dalam mengurangi jumlah percobaan yang diperlukan untuk menemukan hyperparameter terbaik.

Evaluasi Model setelah Hyperparameter Tuning

Setelah proses hyperparameter tuning selesai dan kita telah menemukan kombinasi hyperparameter optimal, langkah berikutnya yang sangat penting adalah melakukan evaluasi model. Evaluasi model bertujuan untuk mengukur performa model setelah tuning dilakukan dan memastikan bahwa model menggeneralisasi dengan baik pada data baru (data yang belum pernah dilihat oleh model). Evaluasi ini tidak hanya mengukur akurasi atau performa di data pelatihan, tetapi juga kemampuan model dalam memprediksi hasil pada data pengujian.

Setelah melakukan evaluasi model setelah hyperparameter tuning, Anda dapat membuat beberapa kesimpulan penting yang nantinya akan berguna untuk membantu membuat keputusan. Berikut beberapa hal yang dapat membantu Anda membuat kesimpulan.

- Apakah hyperparameter tuning meningkatkan performa model? Bandingkan metrik performa sebelum dan sesudah tuning untuk memastikan bahwa tuning memberikan peningkatan signifikan.
- Apakah model overfitting atau underfitting? Pastikan bahwa model bekerja baik pada data uji dan tidak hanya pada data latih.
- Apakah ada metrik lain yang harus dipertimbangkan? Terkadang, akurasi saja tidak cukup. Gunakan metrik lain seperti precision, recall, atau F1-score untuk mendapatkan gambaran performa model yang lebih lengkap.
- Evaluasi yang menyeluruh memastikan bahwa model yang dihasilkan setelah tuning tidak hanya bekerja baik pada dataset pelatihan, tetapi juga dapat menggeneralisasi dengan baik pada data baru.

